

TD 1 (Singleton, Adapter, Decorator)

Exercice 1 L'API java contient une classe nommée WindowAdapter avec l'implémentation suivante :

```
public abstract class WindowAdapter implements WindowListener {  
    public void windowOpened(WindowEvent e) {}  
    public void windowClosing(WindowEvent e) {}  
    public void windowClosed(WindowEvent e) {}  
    public void windowIconified(WindowEvent e) {}  
    public void windowDeiconified(WindowEvent e) {}  
    public void windowActivated(WindowEvent e) {}  
    public void windowDeactivated(WindowEvent e) {} }  
}
```

Question: est-ce que WindowAdapter est un exemple ou une instance du Pattern Adapter?

Exercice 2 Créer le diagramme puis l'implémentation d'une classe Logger(journal) qui ne permettra la création que d'un seul objet log

Exercice 3 Dans la 1ere version de Java, les types agrégats (Vector, HashTable, Stack..) retournaient un itérateur de type Enumeration ; à partir de Java 1.2, de nouveaux agrégats ont été rajoutés, tels les LinkedList, et un nouveau itérateur a été introduit : Iterator.

Les anciens codes qui travaillaient avec Enumeration sont du coup incompatibles avec les LinkedLists. Remédiez à ce problème en écrivant une classe adaptateur : IterationAdapter qui convertit un itérateur de type Iterator pour qu'il puisse travailler avec un code s'attendant à un itérateur de type Enumeration.

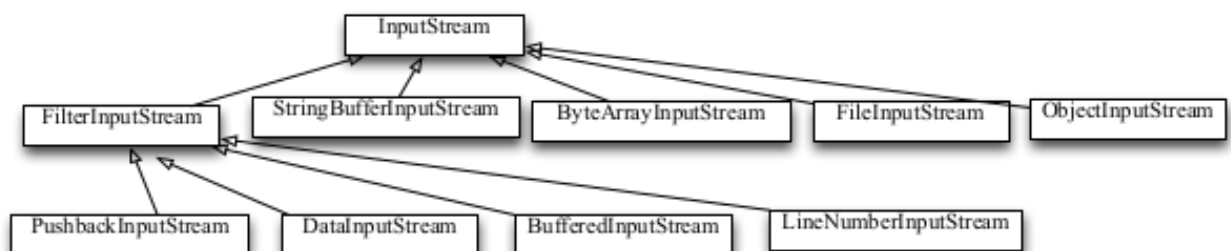
```
import java.util.*;  
public class AdapterExample {  
    public static void main(String[] args) {  
        LinkedList container = new LinkedList();  
        container.add(new Integer(1));  
        container.add(new Integer(2));  
        Iterator iterator =  
            container.iterator();  
        IterationAdapter iterationAdapter = new IterationAdapter(iterator);  
        client(iterationAdapter);  
    }  
    // ce client ne peut appeler que l'ancienne interface Enumeration  
    public static void client(Enumeration e) {  
        while (e.hasMoreElements()) { System.out.println(e.nextElement()); }  
    }  
}  
// Implementer cette classe:  
class IterationAdapter . . .
```

Iterator
hasNext(): Boolean next(): Object remove(): void

<<interface>> Enumeration
hasMoreElements(): boolean nextElement(): Object

Exercice 4

Soit le diagramme de classe suivant, identifiez-y les composants du pattern Decorator



Exercice 5 soit les classes suivantes : TextView, Bordure, BarresDefilantes ; un objet TextView affiche du texte dans une fenêtre, lorsque le texte dépasse la vue écran, on rajoute une barres de défilement, si l'on veut garnir la page , on lui rajoute une bordure.

1. Dessiner le pattern Decorator correspondant

2. Ecrire le code java correspondant

Exercice 6 Créez un fichier(FilesWriter), compressez ses données(compressingStream) puis crypter ses données (EncryptStream) ;
On voudrait pour un autre fichier le crypter, le compresser, puis le crypter une seconde fois.

Exercice 7

Pour communiquer en java entre deux machines, on utilise les sockets. Via les sockets, on obtient un flux(Stream) entrant pour la lecture via la méthode getInputStream() ; ou sortant pour l'écriture avec : getOutputStream(). Pour la lecture, on enveloppe un objet de la classe InputStreamReader, par un décorateur : BufferedReader(ces instructions valent du coté client comme du coté serveurs).

Etant donné l'instruction de création de socket suivante pour un client :

```
Socket cs=new Socket("localhost",9090) ;
```

Construire le Stream qui va lire l'information émise par le serveur(utiliser les 2 classes ci-dessus + la méthode).

De meme, Construire le Stream qui va écrire l'information émise vers le client (par le serveur): on envelope le flux sortant par la classe PrintWriter.